

CS 312 — Assignment 2: Hyperparameter Scaling

Sandra Yang (aleyang)

Setup. Code: `experiments/a2/` on branch `a2-experiments` of the fork `birdhumming/assignments`. Runs: W&B project `assignments`, entity `aleyang-stanford-university`, tag `a2`. We trained 198 new runs, about 28 GPU hours on H100s out of the 48 hour budget. The 78 supplied source runs in `provided_sweeps.csv` were fitted, never retrained. Problem 3.1 ran on CPU. Every prediction below was written down, in `p32_predictions.md` or the launcher manifest, before the matching target run started.

How to read the numbers. Unless stated otherwise, “loss” is the final validation loss in nats of the course d8 model: 8 layers, width 512, batch 64, AdamW with $\beta = (0.9, 0.95)$, weight decay 0.1, 1% warmup then linear decay. Assignment 1 measured the seed to seed standard deviation of this model at about 0.002. So a gap below 0.005 is noise, a gap of 0.01 is real but small, and a gap of 0.1 means two different regimes. lr^* is the minimum of a quadratic fitted in log learning rate (jointly in log learning rate and log weight decay for Problem 2). “Best sampled” is the lowest measured point. D is the number of training tokens and B the batch size.

1 When can training on less data guide large-scale runs?

(a) Small budgets (supplied runs)

tokens	lr 0.0015	lr 0.003	lr 0.006	fitted lr^*	fitted loss	curvature
153.6M	3.2442	3.2291	3.3056	0.00238	3.2240	0.095
307.2M	3.0709	3.0562	3.0715	0.00298	3.0562	0.031
614.4M	2.9473	2.9256	2.9408	0.00319	2.9255	0.038

A power law through the three optima gives $\text{lr}^* = 4.4 \times 10^{-5} D^{+0.21}$ ($R^2 = 0.91$). The optimum rises slowly with budget. At the same time the loss curve flattens: the curvature drops from 0.095 to 0.038, so missing the optimum costs less as the budget grows (Figure 1, left and middle).

(b) Predict before increasing the budget (supplied runs)

tokens	predicted lr^* (small-3 rule)	fitted lr^* after looking	fitted loss	best sampled
1.2288B	0.00380	0.00388	2.8368	0.003 (2.8378)
1.8432B	0.00414	0.00343	2.7964	0.003 (2.7967)
2.4576B	0.00440	0.00291	2.7682	0.003 (2.7683)

The trend does not continue. The optimum peaks near 1.2B tokens and then falls, so the small budget rule overpredicts by 20 to 50% at 1.8B and 2.5B tokens. This barely matters for the loss. Learning rate 0.003 is the best sampled point at every budget, and the curvature keeps falling (to 0.014–0.018), so a 40% miss costs about 0.001.

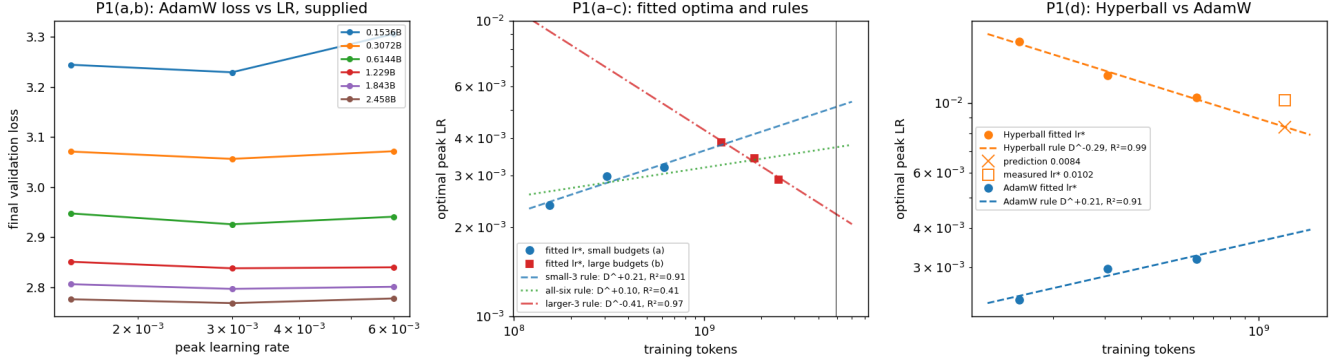


Figure 1: Left: supplied AdamW loss–LR curves. Middle: fitted optima with the three candidate rules; the vertical line marks the 4.9152B target. Right: Hyperball vs. AdamW optima and rules, with the P1(d) prediction (×) and measured optimum (□).

(c) Two scaling rules tested at 4.9152B tokens (5 new runs, ≈ 74 min each)

rule	law	R^2	predicted lr^* at 4.9152B
all six budgets	$\text{lr}^* \propto D^{+0.10}$	0.41	0.00373
three largest budgets	$\text{lr}^* \propto D^{-0.41}$	0.97	0.00223

Prediction, written before running: the fit through the three largest budgets gives the lower loss. The small budgets are in a different regime, where the optimum is still rising, and they bias the slope. An R^2 of 0.41 says the six point data is not a power law at all.

lr at 4.9152B	val loss	gap to best sampled
0.0015	2.7176	+0.0007
0.00223 (larger-three rule)	2.7169	0
0.003 (carry over the 2.5B best)	2.7176	+0.0007
0.00373 (all-six rule)	2.7185	+0.0017
0.006	2.7273	+0.0105

The fitted optimum at 4.9B tokens is 0.00214 (loss 2.7169). The three largest rule landed on it. The all six rule was $1.7\times$ too high and cost 0.0017. Simply reusing 0.003 from the 2.5B run cost 0.0007.

Two things follow. First, small budget tuning does tell you where the flat region is, and a fit over the right regime (the three large budgets, extrapolated by one doubling) predicted the 4.9B optimum almost exactly. Second, the fit was worth very little here. It beat carrying over the previous best learning rate by 0.0007, a third of the seed noise, because the loss curve is nearly flat at this budget.

On the choice of points: using all six lowers the variance of the fit but adds bias from a regime where the optimum was still rising. When the trend changes direction, choosing the right regime matters more than having more points, and a low R^2 is the fit telling you not to extrapolate it. One doubling of extrapolation was safe; the small three rule extrapolated two doublings and was 50% off.

(d) Hyperball versus AdamW (4 new runs at 1.2288B tokens)

Hyperball takes the Adam direction, rescales each weight matrix after every step to keep its initial Frobenius norm, and uses no weight decay. We fitted the supplied sweeps (8 learning rates per budget) the same way as above.

tokens	Hyperball lr*	Hyperball loss*	AdamW lr*	AdamW loss*
153.6M	0.0156	3.2009	0.00238	3.2240
307.2M	0.0122	3.0319	0.00298	3.0562
614.4M	0.0104	2.9176	0.00319	2.9255

Power laws: Hyperball $\text{lr}^* \propto D^{-0.29}$ ($R^2 = 0.985$), AdamW $\text{lr}^* \propto D^{+0.21}$ ($R^2 = 0.91$). Prediction, recorded before running: Hyperball lr^* at 1.2288B tokens is 0.00839.

Hyperball lr at 1.2288B	val loss
0.006	2.8437
0.00839 (predicted)	2.8291
0.01	2.8271
0.015	2.8358

The fitted optimum is 0.0102 (loss 2.8270). The prediction was 18% low and cost 0.002, one seed standard deviation. Hyperball’s best loss (2.8271) also beats AdamW’s best at this budget (2.8378) by 0.011.

Hyperball’s optimum moves more predictably than AdamW’s: a clean, monotone decrease with a tight power law instead of a wobble. The reason is mechanical. With each weight matrix pinned to its initial norm, the effective step is $\eta \|W\|/\|U\|$ and the learning rate is the only knob. In AdamW the effective step depends on a weight norm that the learning rate and weight decay move together during training.

Predictable is not the same as precise. Three source budgets spanning $4\times$ pin the exponent to roughly ± 0.1 , and one more doubling turns that into a 10 to 20% error in the predicted learning rate, which is what we saw. Three points give no estimate of their own error, so a three point extrapolation should be read as “within a factor of 1.5” and swept around.

2 How should learning rate and weight decay scale together?

(a) Joint fits (supplied 3×3 grids)

We fit a quadratic in $(x, y) = (\log \text{lr}, \log \text{wd})$ to each budget’s grid. All four fits are proper minima (positive definite Hessian). Figure 2 shows the contours.

tokens	fitted lr*	fitted wd*	lr* · wd*	fitted loss	best sampled (lr, wd) → loss	P1 best	gain from wd
153.6M	0.00186	0.847	1.57×10^{-3}	3.1766	(0.0015, 1.6) → 3.1844	3.2291	0.045
307.2M	0.00160	0.519	8.33×10^{-4}	3.0309	(0.0015, 0.8) → 3.0282	3.0562	0.028
614.4M	0.00247	0.258	6.37×10^{-4}	2.9175	(0.003, 0.2) → 2.9178	2.9256	0.008
1.2288B	0.00270	0.162	4.37×10^{-4}	2.8349	(0.003, 0.2) → 2.8361	2.8378	0.002

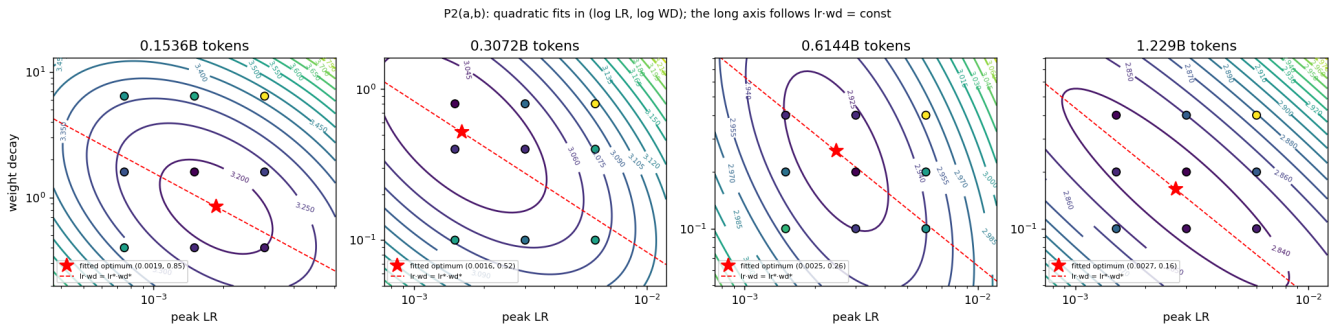


Figure 2: Quadratic fits per budget on log–log axes; dots are supplied runs (colour = loss), the star is the fitted optimum, the dashed line is $\text{lr} \cdot \text{wd} = \text{lr}^* \cdot \text{wd}^*$. The long axis of every ellipse runs along the dashed line.

The optimal learning rate wanders ($0.0019 \rightarrow 0.0016 \rightarrow 0.0025 \rightarrow 0.0027$), as in Problem 1. The optimal weight decay falls steadily, from 0.85 to 0.16, a factor of 5 over $8\times$ more tokens. The gain from tuning weight decay shrinks from 0.045 at 153.6M tokens to 0.002, which is noise, at 1.2B (Figure 3, right).

(b) From coupling to a joint scaling law

quantity	power law in tokens	R^2
lr^*	$\propto D^{+0.22}$	0.68
wd^*	$\propto D^{-0.82}$	0.994
$lr^* \cdot wd^*$	$\propto D^{-0.59}$	0.967

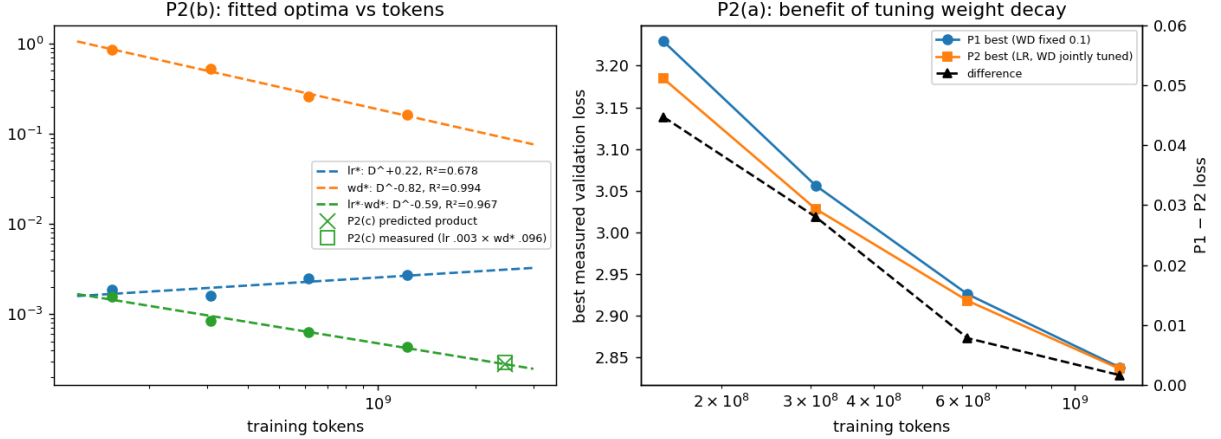


Figure 3: Left: fitted lr^* , wd^* , and their product vs. tokens with power laws; the P2(c) prediction ($\times$) and the measured optimum at 2.4576B ($\square$). Right: best measured loss with WD fixed at 0.1 (P1) vs. jointly tuned (P2), and the difference.

One hyperparameter does compensate for the other. On every contour plot the valley runs along $lr \cdot wd = \text{const}$: raising one and lowering the other by the same factor barely changes the loss, while moving across that line changes it a lot. The product follows a tight power law ($R^2 = 0.97$). Weight decay alone fits even better here (0.994), but only because the learning rate was nearly constant across these budgets. The product is the quantity with a physical meaning. AdamW shrinks the weights by a factor $(1 - lr \cdot wd)$ every step, so $1/(lr \cdot wd)$ is the number of updates the weights remember.

(c) Test of the joint rule at 2.4576B tokens (5 new runs)

The product law fitted to the four source budgets predicts $lr^* \cdot wd^* = 2.78 \times 10^{-4}$ at 2.4576B tokens. At the peak learning rate 0.003 that gives a **predicted weight decay of 0.0927**.

lr	wd	lr·wd	val loss	gap to prediction
0.0015	1.6 — baseline (i): best 153.6M pair, untuned	2.4×10^{-3}	2.8517	+0.084
0.003	0.05	1.5×10^{-4}	2.7715	+0.004
0.003	0.0927 (prediction)	2.8×10^{-4}	2.7680	—
0.003	0.1 — baseline (ii): grid best	3.0×10^{-4}	2.7671	−0.001
0.003	0.2	6.0×10^{-4}	2.7725	+0.005

A quadratic in $\log wd$ at lr 0.003 puts the optimum at 0.096 with loss 2.7675. The prediction is within noise of the tuned grid best (−0.001) and 0.084 better than carrying over the small budget pair.

The product rule works as a recipe: hold the learning rate anywhere in its flat region and set the weight decay to the predicted product divided by that learning rate. The valley is long and narrow along the product direction, so fixing one coordinate and solving for the other loses nothing. The product falls like $D^{-0.6}$, so the weight memory in steps

grows like $D^{0.6}$, slower than the run length. That is why the benefit of tuning weight decay shrinks with budget: at large budgets the default 0.1 is already close to optimal.

The expensive mistake is baseline (i). A weight decay of 1.6 that was right at 153.6M tokens is $17\times$ too large at 2.5B tokens and costs 0.084, more than any learning rate mistake in Problem 1.

3 Batch-size scaling

3.1 What does the noisy quadratic model predict? (CPU)

Setup: $f(w) = \frac{1}{2}w^\top Hw$ with $H = \text{diag}(1, 10)$, gradient noise covariance $\sigma^2 I$, $w_0 \sim \mathcal{N}(0, 1) \oplus \mathcal{N}(0, 0.1)$, $N = 8192$ samples per epoch so the number of updates is N/B , and 1024 Monte Carlo samples per loss estimate. The source batches are 1 to 64. We fit $\text{lr}^* \propto B^p$, predict lr^* at $B = 256$, and test the prediction with a sweep at 256.

(a)–(b) Scaling exponent by optimizer ($\sigma = 1$)

optimizer	exponent p (B 1–64)	predicted $\text{lr}^*(256)$	measured $\text{lr}^*(256)$	loss at predicted	tuned loss	gap
SGD	0.99	0.184	0.123	0.00237	0.00073	+0.0016
RMSProp	0.90	0.113	0.094	0.0151	0.0121	+0.0031
Adam	0.92	0.125	0.156	0.0149	0.0144	+0.0005

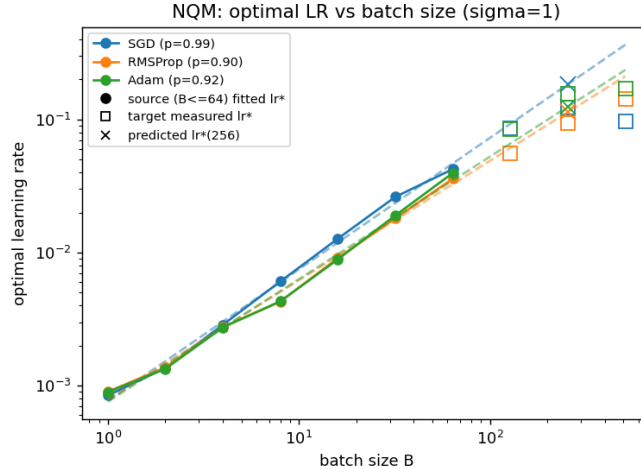


Figure 4: Optimal LR vs. batch for SGD, RMSProp, Adam ($\sigma = 1$): source fits (dots, dashed), prediction at 256 ($\times$), measured optimum ($\square$).

At low noise all three optimizers scale almost linearly with batch. The SGD rule breaks exactly where it should: the predicted 0.184 sits at the stability edge of the stiff direction ($\text{lr} < 2/10 = 0.2$), so the measured optimum backs off to 0.123. Adam and RMSProp have similar exponents, but Adam transfers better (gap 0.0005 versus 0.0031). Same exponent, different curve width, so a matching exponent does not guarantee matching transfer accuracy.

Tuned SGD loss by batch goes 0.00037 ($B=1$) $\rightarrow$ 0.00032 ($B=8\text{--}16$) $\rightarrow$ 0.00039 ($B=64$) $\rightarrow$ 0.00073 ($B=256$). The best batch is around 8 to 16. Past that the loss gets worse, because at fixed N the number of updates is what you pay with and $B = 256$ leaves only 32 of them. The rule holds while noise dominates the error. Once the budget is a handful of steps, the stability ceiling and the step count take over.

Prediction for the language model, made before P3.2: the optimal learning rate rises with batch, close to linearly from batch 8 to 64, then flattens by 128 to 256 where per step noise stops being the bottleneck.

(c) Noise level

σ	RMSProp exponent (2-D)	Adam exponent (2-D)	RMSProp (scalar $f = w^2/2$)	Adam (scalar)
1	0.91	0.92	0.94	0.93
10	0.55	0.54	0.57	0.55
100	0.45	0.49	0.50	0.48
300	0.53	0.49	0.40	0.45

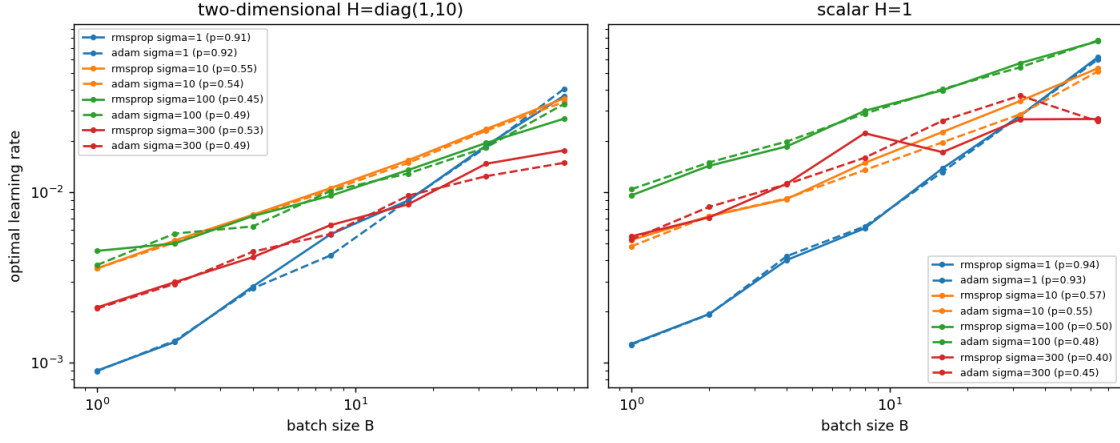


Figure 5: Optimal LR vs. batch by noise level for the 2-D (left) and scalar (right) models; exponents in the legend.

Signal to noise does change the rule. Adam’s second moment is $v \approx g^2 + \sigma^2/B$. When the gradient dominates, v does not depend on batch, $g/\sqrt{v}$ behaves like SGD, and $\text{lr}^* \propto B$. When noise dominates, $\sqrt{v} \propto \sigma/\sqrt{B}$ already scales the update by $\sqrt{B}$, so the learning rate only needs to supply another $\sqrt{B}$: exponent $\frac{1}{2}$. The 2-D and scalar models agree, so this comes from second moment normalisation, not from curvature. Curvature decides something else: where the linear rule breaks. The stiff direction imposes $\text{lr} < 2/\lambda_{\max}$ in 2-D, and the scalar model has no such ceiling.

(d) Momentum at small and large batch

setting	$B = 16$ (512 updates): $\text{lr}^* \rightarrow \text{loss}$	$B = 256$ (32 updates): $\text{lr}^* \rightarrow \text{loss}$
SGD, $\mu = 0$	0.0126 $\rightarrow$ 0.00035	0.124 $\rightarrow$ 0.00067
SGD, $\mu = 0.9$	0.00127 $\rightarrow$ 0.00032	0.0083 $\rightarrow$ 0.0041
Adam, $\beta_1 = 0$	0.0089 $\rightarrow$ 0.00104	0.093 $\rightarrow$ 0.0123
Adam, $\beta_1 = 0.5$	0.0089 $\rightarrow$ 0.00082	0.133 $\rightarrow$ 0.0035
Adam, $\beta_1 = 0.9$	0.0090 $\rightarrow$ 0.00083	0.156 $\rightarrow$ 0.0143
Adam, $\beta_1 = 0.95$	0.0090 $\rightarrow$ 0.00094	0.188 $\rightarrow$ 0.050
Adam, $\beta_1 = 0.99$	0.018 $\rightarrow$ 0.0127	0.064 $\rightarrow$ 0.134

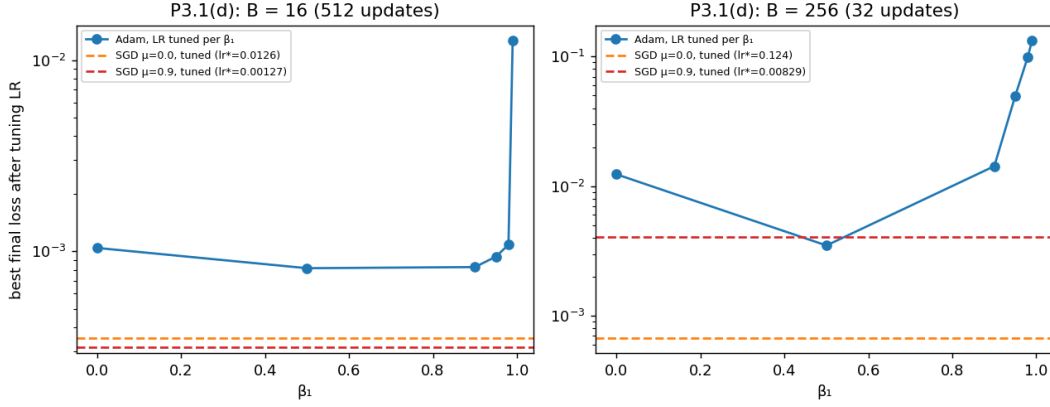


Figure 6: Best loss after tuning the LR at each β_1 , with tuned SGD with and without momentum as reference lines.

Momentum helps a little at small batch: SGD goes from 0.00035 to 0.00032, and Adam with β_1 between 0.5 and 0.9 beats $\beta_1 = 0$ by 20%. It hurts at large batch even after the learning rate is retuned: SGD with $\mu = 0.9$ is $6\times$ worse, and Adam with $\beta_1 \geq 0.9$ is worse than 0.5. The tuned SGD learning rate drops $15\times$ with momentum at $B = 256$ ($0.124 \rightarrow 0.0083$) because the effective step $\text{lr}/(1 - \mu)$ has to be cut to stop the stiff direction from overshooting.

Momentum averages gradients over about $1/(1 - \beta_1)$ updates. That is noise reduction, which is also what a bigger batch does, so the benefit shrinks as the batch grows. The cost is lag. At $B = 256$ a $\beta_1 = 0.9$ window of about 10 updates is a third of the 32 update run. Our expectation for the language model, written before P3.2(c): $\beta_1 = 0.9$ helps slightly at batch 8 and matters little at batch 256, because the language model run has thousands of updates rather than 32 and the lag cost should not appear. This turned out half right.

3.2 What should scale when batch size changes? (language model)

Setup: the d8 model, 614.4M tokens, AdamW, 1% warmup then linear decay, microbatch $\min(B, 64)$. We ran 15 new source runs at batch 8, 16 and 32 (three learning rates at weight decay 0.1, plus weight decay 0.05 and 0.2 at learning rate 0.0015) and reused the supplied batch 64 curves from Problems 1 and 2 as the fourth source. After writing the predictions down, we ran 13 target runs at batch 128 and 256 (trimmed to four points per curve) and 6 β_1 controls. Each point is one run.

(a) Optimal learning rate and loss vs. batch size (weight decay 0.1)

batch	LRs tried	best sampled	fitted lr^*	loss at lr^*	updates	run time
8	0.0004 / 0.00075 / 0.0015	0.0015	0.00109	2.934	75,000	16 min
16	0.00075 / 0.0015 / 0.003	0.0015	0.00174	2.922	37,500	13 min
32	0.0015 / 0.003 / 0.006	0.003	0.00329	2.919	18,750	12 min
64 (supplied)	0.0015 / 0.003 / 0.006	0.003	0.00319	2.925	9,375	—

The source fit is $\text{lr}^* \approx 0.00037 B^{0.56}$. The optimum rises roughly like $\sqrt{B}$ from 8 to 32, not linearly as the low noise NQM predicted and not flat either, then stops between 32 and 64. The best loss does not depend on batch between 16 and 64 (2.92 ± 0.003). Batch 8 is 0.012 worse despite 75,000 updates, so its penalty comes from too few tokens per update, not too few updates.

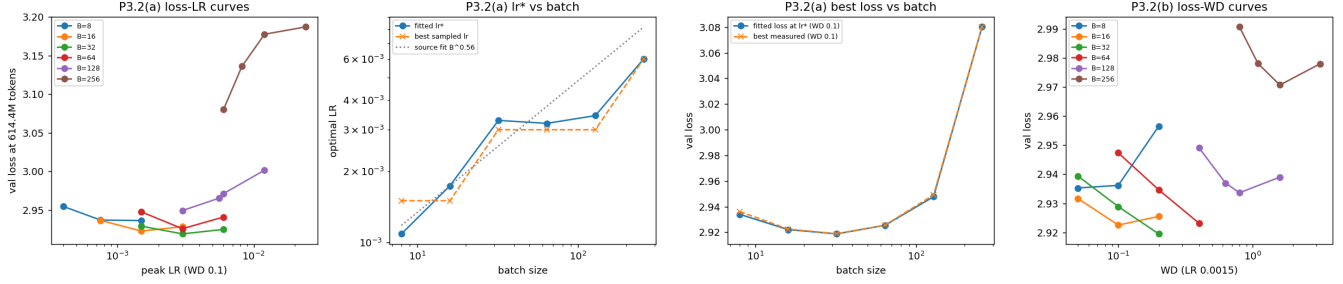


Figure 7: Left to right: loss-LR curves at wd 0.1 for all batches (targets included); fitted and best-sampled lr^* vs. batch with the source $B^{0.56}$ law; best loss vs. batch at wd 0.1; loss-WD curves at lr 0.0015.

(b) Two scaling hypotheses: scale LR at fixed WD, or scale WD at fixed LR 0.0015

Source side at lr 0.0015:

batch	WDs tried	best sampled wd	fitted wd*
8	0.05 / 0.1 / 0.2	0.05	0.068
16	0.05 / 0.1 / 0.2	0.1	0.12
32	0.05 / 0.1 / 0.2	0.2	≥ 0.2 (grid edge)
64 (supplied)	0.1 / 0.2 / 0.4	0.4	≥ 0.4 (grid edge)

The weight decay optimum roughly doubles each time the batch doubles. Two of the source optima sit on the grid edge, so the exponent is poorly determined: the interior fit gives $wd^* \propto B^{0.80}$ (0.63 at 128, 1.1 at 256), and a proportional rule gives 0.95 and 1.9. We used the interior fit for the predicted point and bracketed it.

Predictions recorded before launch. Hypothesis (i), weight decay fixed at 0.1: lr 0.0056 at $B = 128$ and 0.0082 at $B = 256$. Hypothesis (ii), learning rate fixed at 0.0015: wd 0.63 at 128 and 1.1 at 256. **We predicted that hypothesis (i) would win.** Scaling the learning rate keeps the per update step matched to the lower gradient noise. Hypothesis (ii) keeps a small step and compensates with a decay so strong (a memory of 500 to 1,000 updates out of 2,300 to 4,700) that we expected it to erase learning.

target batch	hypothesis	points (lr or wd $\rightarrow$ loss)	best
128 (4,688 updates)	(i) wd 0.1, vary lr	0.003 $\rightarrow$ 2.949 ; 0.0056 (pred) $\rightarrow$ 2.965; 0.006 $\rightarrow$ 2.971; 0.012 $\rightarrow$ 3.002	2.949
	(ii) lr 0.0015, vary wd	0.4 $\rightarrow$ 2.949; 0.63 (pred) $\rightarrow$ 2.937; 0.8 $\rightarrow$ 2.934 ; 1.6 $\rightarrow$ 2.939	2.934
256 (2,344 updates)	(i) wd 0.1, vary lr	0.006 $\rightarrow$ 3.080 ; 0.0082 (pred) $\rightarrow$ 3.136; 0.012 $\rightarrow$ 3.178; 0.024 $\rightarrow$ 3.187	3.080
	(ii) lr 0.0015, vary wd	0.8 $\rightarrow$ 2.991; 1.1 (pred) $\rightarrow$ 2.978; 1.6 $\rightarrow$ 2.971 ; 3.2 $\rightarrow$ 2.978	2.971

Hypothesis (ii) wins at both targets: 2.934 versus 2.949 at batch 128 (a gap of 0.015) and 2.971 versus 3.080 at batch 256 (0.11). Our prediction was wrong, for three reasons.

1. The learning rate optimum stopped rising. It grew like $B^{0.56}$ from 8 to 32, stayed flat from 32 to 64 (0.0033 $\rightarrow$ 0.0032), was 0.0034 again at 128, and at 256 the best point is the *lowest* learning rate we tried. Extrapolating the source law to 0.0056 and 0.0082 overshot, and the loss curve is steep on the high side: 0.012 at batch 256 costs 0.1.
2. At fixed weight decay 0.1, large batches simply lose. The best batch 256 point (3.080) is 0.16 worse than batch 32. With 2,344 updates instead of 18,750, a fixed per update decay means an $8\times$ longer memory in tokens, and the model carries early, badly fitted weights to the end.
3. The weight decay optimum doubled every time the batch doubled: best sampled 0.1 (16), 0.2 (32), 0.4 (64), 0.8 (128), 1.6 (256), with fitted target optima of 0.95 and 1.88. That is $wd^* \propto B$. The proportional rule (0.95 / 1.91) was right to 2%; the interior power law undershot because two of its four points were lower bounds. Scaling weight decay with batch keeps $lr \cdot wd \cdot (\text{updates per token})$, the decay timescale measured in tokens, constant. That is the same quantity Problem 2 found mattered.

Even with the right weight decay, batch 256 is 0.05 worse than batch 32 at this token budget (2.971 vs. 2.919), and batch 128 is 0.015 worse. Fewer updates cannot be fully bought back with either knob. This agrees with the NQM only in the small batch range, where the exponent of about $\frac{1}{2}$ matches its high noise regime. The NQM has no critical batch size and no weight decay, so it cannot predict the flattening or the $\text{wd} \propto B$ rule.

(c) Momentum ablation

We held each batch’s best measured pair fixed (batch 8: lr 0.0015, wd 0.05; batch 256: lr 0.0015, wd 1.6), with $\beta_2 = 0.95$ and 614.4M tokens, and varied β_1 over $\{0, 0.5, 0.98\}$ plus the existing $\beta_1 = 0.9$ run.

β_1	batch 8 (75,000 updates)	batch 256 (2,344 updates)
0 (no momentum)	2.978	3.198
0.5	2.963	3.060
0.9 (tuned runs)	2.935	2.971
0.98	2.934	3.025
cost of removing momentum (0.9 $\rightarrow$ 0)	+0.043	+0.228

Momentum matters more at large batch, not less, which is the opposite of the NQM’s call. Dropping β_1 to 0 costs 0.04 at batch 8 and 0.23 at batch 256, five times as much. Raising it to 0.98 is free at batch 8 (2.934 vs. 2.935) but costs 0.054 at batch 256, so at large batch the optimum is pinned near 0.9 from both sides. One caveat: in P3.1(d) we retuned the learning rate for each β_1 , but here it is held at the $\beta_1 = 0.9$ optimum, so the $\beta_1 = 0$ runs may be slightly undertuned. The gap of 0.23 is far beyond what a retune could recover given the shape of the batch 256 loss curve in P3.2(a).

Our reading, which is an interpretation rather than a measured mechanism: the NQM’s “momentum hurts at large batch” result was lag in a 32 update run, where a $\beta_1 = 0.9$ window is a third of training. The language model at batch 256 has 2,344 updates, so that lag is negligible at 0.9 and only shows up at 0.98 (a window of about 50 updates, 2% of the run, costing 0.054). What remains is the benefit. Adam with $\beta_1 = 0$ is RMSProp: every coordinate moves by about lr per update regardless of signal to noise. At batch 8 the 75,000 small steps under a long decaying schedule average that noise out on their own. At batch 256 each of the 2,344 steps is $32\times$ more consequential, and momentum is the only noise reduction left.

(d) What does the NQM explain?

quantity	NQM prediction (P3.1)	language model (P3.2)	verdict
lr* vs. batch, low noise	$\propto B^{0.9-1.0}$	$\propto B^{0.56}$ for $8 \rightarrow 32$, then flat, then <i>lower</i> at 128–256 (0.003 $\rightarrow$ 2.949 beats 0.0056 $\rightarrow$ 2.965; 0.006 $\rightarrow$ 3.080 beats 0.0082 $\rightarrow$ 3.136)	fails above batch ≈ 32 : the NQM has no critical batch size where more sequences per update stop reducing useful noise
lr* vs. batch, high noise	$\propto B^{0.5}$	$B^{0.56}$ in the small-batch range	agrees where the LM is noise-limited
best loss vs. batch	at fixed N : bigger batch better until the stability/step-count ceiling ($B \approx 8$ –16 best)	fixed tokens: 2.934 (8) $\rightarrow$ 2.919 (32) $\rightarrow$ 2.925 (64) $\rightarrow$ 2.934 (128, retuned wd) $\rightarrow$ 2.971 (256, retuned wd)	qualitatively agrees: an interior best batch, a penalty from the update count beyond it
weight decay vs. batch	not modelled	$\text{wd}^* \propto B$, 0.1 $\rightarrow$ 1.6 over 16 $\rightarrow$ 256; keeping wd fixed costs 0.11 at batch 256	an effect the NQM does not model , not a failed prediction
momentum (β_1)	helps slightly at small batch, hurts at large batch even with the LR retuned	0.9 $\rightarrow$ 0 costs 0.043 at batch 8 and 0.228 at batch 256; 0.98 $\approx$ 0.9 at batch 8 but costs 0.054 at batch 256	small-batch half agrees; large-batch half fails : the NQM’s harm was lag in a 32-update run

There are two currencies here: tokens, which you pay for, and updates, which move the weights. Doubling the batch halves the updates per token, and every hyperparameter defined per update has to be rescaled. The learning rate is

per update, but once noise stops being the limit it is capped by curvature. That cap is a critical batch size, around 32 to 64 here. The NQM’s low noise regime has no such thing because its noise never stops averaging down. Weight decay is also per update: $1/(\text{lr} \cdot \text{wd})$ updates of memory. Keeping it fixed while halving updates per token doubles the memory in tokens, so $\text{wd}^* \propto B$ means “keep the memory constant in tokens”. It is the Problem 2 product rule with batch as a third factor.

One change to the NQM would fix the biggest disagreement: add a per coordinate noise floor that does not shrink with batch (equivalently, a finite number of useful directions). Then gradient noise reduction saturates, the optimal step stops growing past a critical batch, and fixed token loss gets worse beyond it. Adding an L_2 term with a timescale in updates would also reproduce $\text{wd}^* \propto B$. The practical rule for this model: when changing batch size at fixed tokens, keep the learning rate near its batch 32 to 64 value, scale weight decay in proportion to the batch, keep $\beta_1 = 0.9$, and expect a loss penalty at batch 128 and above anyway.

4 Width and depth scaling

4.0 Scaling rules from alignment assumptions

Setup: width n , reference width n_0 , $m = n/n_0$. Each matrix is used as $n^{-a}W$, initialised with i.i.d. entries of variance n^{-2b} , and updated by $\Delta W = -\eta n^{-c}U$ with $\|U\|_{\text{RMS}} = \Theta(1)$. $a = 0$ for hidden and input matrices, $c = 0$ for the readout. The alignment ratios are $R(U, x_0) = \Theta(n^\alpha)$, with $\alpha = 1$ if the update direction is aligned with its input (n terms add coherently) and $\alpha = \frac{1}{2}$ if not (a random walk sum), and $S(V_0, \Delta x) = \Theta(n^\omega)$, with $\omega = 1$ aligned and $\frac{1}{2}$ not. We assume $R(U, \Delta x) = O(n^\alpha)$.

(a) Constraints. *Hidden* ($a = 0$, fan-in $k = n$). Order one features at initialisation: $\|Wx_0\| \sim \sqrt{n} n^{-b} = \Theta(1) \Rightarrow b = \frac{1}{2}$ (variance $1/k$), independent of alignment. Order one direct update: $\|\Delta Wx_0\| = \eta n^{-c} R(U, x_0) = \eta n^{\alpha-c} = \Theta(1) \Rightarrow c = \alpha$. Interaction: $\|\Delta W \Delta x\| = O(n^{\alpha-c}) = O(1)$, bounded automatically.

Readout ($c = 0$, transposed $V_0 \in \mathbb{R}^{n \times q}$, response $n^{-a}V^\top x$). Direct update: $\eta n^{\alpha-a} = \Theta(1) \Rightarrow a = \alpha$. Response of the initial readout to the feature change: $n^{-a}S(V_0, \Delta x) n^{-b} = n^{\omega-a-b} = \Theta(1) \Rightarrow b = \omega - \alpha$. Interaction $\eta n^{\alpha-a} = O(1)$, bounded.

Input / embedding (fixed fan-in: a token hits one column, so there is no sum over n). Order one features $\Rightarrow b = 0$ (variance 1). Order one update $\|\Delta W e_j\| = \eta n^{-c} \Rightarrow c = 0$. Alignment plays no role.

alignment (α, ω)	hidden (a, b, c)	readout (a, b, c)	input (a, b, c)
update aligned, readout aligned (1, 1)	$(0, \frac{1}{2}, 1)$	$(1, 0, 0)$	$(0, 0, 0)$
update aligned, readout not (1, $\frac{1}{2}$)	$(0, \frac{1}{2}, 1)$	$(1, -\frac{1}{2}, 0)$	$(0, 0, 0)$
update not aligned, readout aligned ($\frac{1}{2}, 1$)	$(0, \frac{1}{2}, \frac{1}{2})$	$(\frac{1}{2}, \frac{1}{2}, 0)$	$(0, 0, 0)$
neither aligned ($\frac{1}{2}, \frac{1}{2}$)	$(0, \frac{1}{2}, \frac{1}{2})$	$(\frac{1}{2}, 0, 0)$	$(0, 0, 0)$

(b) Rules in $m = n/n_0$. Matching the table at n_0 gives a forward multiplier m^{-a} , an initialisation variance $\sigma_0^2 m^{-2b}$, and a learning rate ηm^{-c} .

layer	forward multiplier	init variance	learning rate
hidden	1	$1/k$	$\eta m^{-\alpha}$
readout	$m^{-\alpha}$	$n_0^{-1} m^{-2(\omega-\alpha)}$	η
embedding	1	1	η

Update alignment α sets how fast hidden learning rates shrink ($1/m$ aligned, $1/\sqrt{m}$ not) and how strongly the readout is damped ($1/m$ vs. $1/\sqrt{m}$). Readout alignment ω only changes the readout initialisation. With $\omega = \alpha$ the variance does not depend on width ($1/n_0$). With $\omega < \alpha$ it would have to *grow* with width (m^{+1}), which is the sign that “aligned updates, unaligned readout” is not a consistent regime. The interaction terms are $O(n^{\alpha-c})$ and $O(n^{\alpha-a})$, bounded in every row once the direct update constraint holds.

μP is the case $(\alpha, \omega) = (1, 1)$: hidden variance $1/k$ and learning rate η/m ; readout variance $1/n_0$, multiplier $1/m$, learning rate η ; embedding variance 1, learning rate η . This is exactly the P4.2 μP column (G/n_0 , $G/\sqrt{n_0}$, $1/m$,

η/m at $n_0 = 512$). Kaiming keeps the readout at $(0, \frac{1}{2}, 0)$ and hidden $c = 0$. Its readout satisfies $a + b = \frac{1}{2} = \omega$ only for an unaligned readout, and with aligned Adam updates its first step moves features by $\Theta(n)$ at fixed η , so its tuned base learning rate must fall like $1/n$. That is the prediction P4.1 tests.

4.1 Five-step Transformer stress test

Five Adam updates, constant learning rate, no warmup, no weight decay, the same five batches for every configuration, and fixed probe inputs for all measurements.

(a) Width transfer (36 runs; depth 2, reference width 512, FP32)

Six learning rates per cell (0.00025 to 0.008, doubling), plus 0.0000625 and 0.000125 for wide Kaiming once its optimum fell off the grid. Prediction before running: Kaiming’s best base learning rate falls like $1/\text{width}$ (about 0.00025 at 2560 and 0.000125 at 5120), while μP ’s stays near its width 640 value.

policy	width	best sampled lr	fitted lr*	loss at best	at lr 0.00025	at 0.002	at 0.008
Kaiming	640	0.001	0.0011	7.069	7.885	7.098	8.336
Kaiming	2560	0.000125	0.00015	7.184	7.245	9.235	15.39
Kaiming	5120	0.0000625 (grid edge)	≤ 0.0000625	7.325	8.049	11.13	21.46
μP	640	0.002	0.0015	6.784	8.006	6.784	8.015
μP	2560	0.002	0.0013	6.669	7.710	6.669	8.127
μP	5120	0.002	0.0012	6.700	7.655	6.700	7.968

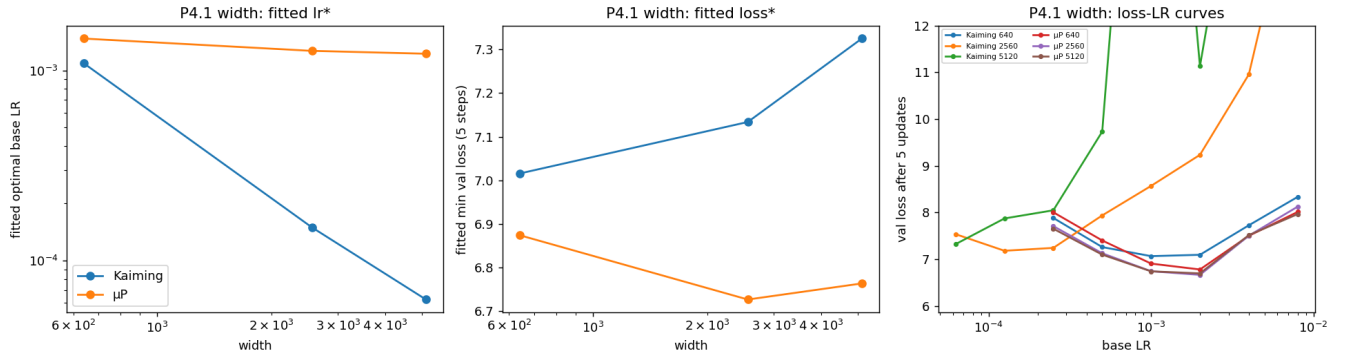


Figure 8: Fitted optimal base LR (left) and fitted minimum five-step loss (middle) vs. width; loss-LR curves (right).

Both predictions held. Kaiming’s optimum fell slightly faster than $1/\text{width}$: $7\times$ from 640 to 2560 and at least $16\times$ by 5120, where the width 640 optimum of 0.001 gives a loss of 22.8. μP ’s best sampled learning rate is 0.002 at every width, and its fitted optimum moves 20% over an $8\times$ change in width. So μP transfers its tuned base learning rate and Kaiming does not. μP also reaches the lower loss at every width (6.78 / 6.67 / 6.70 vs. 7.07 / 7.18 / 7.33), improving with width while Kaiming gets worse.

(b) Width probes (at each configuration’s best sampled LR)

policy	width	logit RMS, step 0 $\rightarrow$ 5	feature movement M_5	ω_{move}	α_{upd} hidden (q, k, v, gate, up, down), step 1 $\rightarrow$ 5	α_{upd} attention-out / readout, step 1 $\rightarrow$ 5
Kaiming	640	1.00 $\rightarrow$ 1.26	1.07	0.501–0.507	0.59– 0.64 $\rightarrow$ 0.64–0.88	0.79–0.82 / 0.63 $\rightarrow$ 0.92
Kaiming	2560	1.00 $\rightarrow$ 1.23	0.85	0.512–0.519	0.65– 0.70 $\rightarrow$ 0.69–0.74	0.83 / 0.69
Kaiming	5120	1.00 $\rightarrow$ 1.33	1.08	0.516–0.528	—	0.85–0.87 / 0.71 $\rightarrow$ 0.89
μP	640	0.89 $\rightarrow$ 1.61	1.12	0.500–0.506	0.59– 0.64 $\rightarrow$ 0.65–0.92	0.79–0.82 / 0.63 $\rightarrow$ 0.92
μP	2560	0.45 $\rightarrow$ 1.45	1.02	0.514–0.517	—	0.83–0.85 / 0.68 $\rightarrow$ 0.93

The residual stream RMS before the final norm grows from 1 to between 3.1 and 3.9 in every case, while normalised features stay at RMS 1.00. Four things stand out.

A tuned learning rate means order one feature movement. At the best learning rate the final normalised features have moved by RMS ≈ 1 after five steps in every configuration, which is the derivation’s definition of a correct step size. μP produces this at the same base learning rate at every width. Kaiming needs a smaller base learning rate as width grows because its hidden updates move features by $\Theta(\text{width})$.

The initial logit scale differs by design. Kaiming’s $1/n$ readout variance gives logit RMS 1 at every width. μP ’s (variance $1/n_0$, multiplier $1/m$) gives $1/\sqrt{m}$: 0.89 at 640, 0.45 at 2560. After five updates both reach 1.2 to 1.6, so in μP the learned part of the readout carries the signal.

Update alignment is partial, not full. α_{upd} is 0.59 to 0.70 at step 1 for the q, k, v, gate, up and down matrices, higher (0.79 to 0.87) for the attention output projection, and rises to 0.7 to 0.95 by step 5. Adam updates are far more aligned than random ($\frac{1}{2}$) but not fully coherent (1). The η/m rule still transferred because slightly over shrinking the hidden learning rate only shifts the optimum within the flat region; that is the 20% drift we see.

Readout to feature change alignment is at the no alignment scale. ω_{move} is 0.50 to 0.53 everywhere, against μP ’s assumed $\omega = 1$. Under μP ’s initialisation $V_0^\top \Delta x$ then scales like $m^{\omega-\alpha-b} = m^{-1/2}$: a bounded, vanishing contribution, which is exactly why μP ’s initial logits shrink with width and the learned readout takes over. The assumption that fails is one the learning rate rule did not need.

(c) Depth transfer (63 runs; width 64, one head, reference depth 2, mixed precision)

Nine learning rates per cell (0.00025 to 0.064, doubling), all on top of the μP width rule, with $r = \text{depth}/2$.

prescription	residual-branch multiplier	hidden LR multiplier	Adam ϵ multiplier
μP (no depth rule)	1	1	1
Depth- μP	$r^{-1/2}$	$r^{-1/2}$	$r^{-1/2}$
CompleteP	$1/r$	1	$1/r$

Prediction before running: CompleteP transfers best, Depth- μP next, plain μP worst, because only CompleteP keeps each block’s contribution to the residual stream independent of depth.

prescription	depth	best sampled lr	fitted lr*	loss at best	loss at lr 0.008	loss at lr 0.032
μ P	2 (same model for all three)	0.016	0.0174	6.858	7.216	7.119
μ P	100	0.016	0.0146	7.060	7.270	7.135
μ P	1000	0.016	0.0138	7.067	7.322	7.281
Depth- μ P	100	0.016	0.0197	6.867	7.260	7.107
Depth- μ P	1000	0.016	0.0170	6.951	7.151	7.028
CompleteP	100	0.016	0.0179	6.816	7.209	7.065
CompleteP	1000	0.016	0.0158	6.863	7.220	7.271

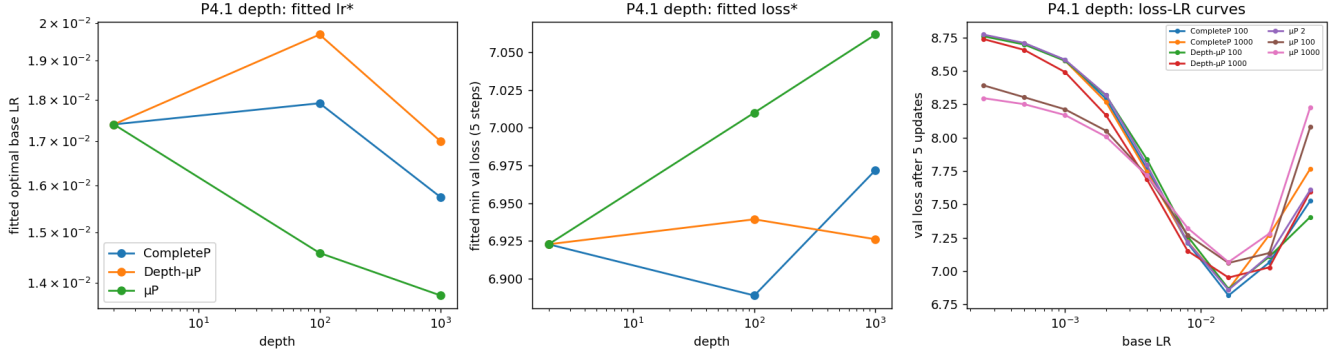


Figure 9: Fitted optimal base LR and fitted minimum loss vs. depth for the three prescriptions; loss-LR curves (right).

At the resolution of a doubling grid, all three prescriptions transfer the base learning rate from depth 2 to depth 1000: the best sampled value is 0.016 in every cell. The fitted optima drift down by 21% (μ P), 14% (Depth- μ P) and 12% (CompleteP) over a $500\times$ change in depth, which is the predicted ranking but a small effect. Loss is where they separate. Plain μ P’s tuned loss gets worse with depth ($6.86 \rightarrow 7.06 \rightarrow 7.07$), while Depth- μ P ($6.87 \rightarrow 6.95$) and CompleteP ($6.82 \rightarrow 6.86$) stay near the depth 2 value, with CompleteP best at both depths. Better transfer does come with lower loss, but the loss gap (0.2) is much larger than the transfer gap (a few percent in learning rate). The depth rules matter more for what a deep model can do in five steps than for where its optimum sits.

(d) Depth probes (best sampled LR 0.016, plus 0.008)

prescription	depth	residual RMS before final norm	branch-output RMS, last block (attn / MLP)	feature movement M_5	ω_{move}
μ P	2	9.6	4.3 / 4.4	1.1	0.50
μ P	100	696 (235 at lr 0.008)	3.4 / 6.7	1.2	0.50
μ P	1000	6378 (2400 at lr 0.008)	3.4 / 6.7	1.25	0.50
Depth- μ P	1000	7.7 (5.0 at lr 0.008)	≈ 1	1.1	0.50
CompleteP	100	6.0	≈ 1	1.1	0.50
CompleteP	1000	6.2 (5.9 at lr 0.008)	≈ 1	1.1	0.50

Normalised activations do hide a growing residual stream. Plain μ P at depth 1000 feeds a residual stream of RMS 6378 into the final RMSNorm, yet the normalised features have RMS 1 and have moved by $M_5 \approx 1.1$ –1.25 like every other configuration, and logits are RMS 1.7 to 2.0 everywhere. The residual RMS grows roughly linearly in depth ($696 \rightarrow 6378$ for $10\times$ depth), not like $\sqrt{\text{depth}}$: after Adam’s first update the branch outputs are coherent and add rather than average. Depth- μ P ($1/\sqrt{r}$) bounds the stream at 7.7 and CompleteP ($1/r$) at 6.2, flat between depth 100 and 1000.

The loss follows from this. In plain μ P the final norm divides by about 6000, so each block’s order one contribution is $1/6000$ of its depth 2 value; 1000 blocks are used as one averaged direction, which is why its loss plateaus at 7.06 while CompleteP’s blocks keep individual leverage (6.82 to 6.86). The learning rate optimum barely moves for any prescription because the base learning rate sets Adam’s per entry step and RMSNorm absorbs the scale; the depth rule changes the quality of the step, not its size. ω_{move} is 0.50 at every depth and prescription. Hidden α_{upd} for query matrices is 0.53 at step 1 at shallow depth but already 0.73 to 0.80 for plain μ P at depth 100 to 1000, which is the coherence behind the linear growth showing up directly.

The derivation’s two assumptions fare differently. $\alpha = 1$ is approximately right (0.6 to 0.9 and rising); $\omega = 1$ is wrong everywhere (0.5). The rules that matter, the hidden learning rate η/m and the readout multiplier $1/m$, depend on α , so μP transfers. The rule that depends on ω , the readout initialisation, only decides whether the initial readout contributes, and it does not. For depth, “transfers the learning rate” and “keeps the residual stream bounded” are different properties. All three prescriptions have the first over a doubling grid. Only Depth- μP and CompleteP have the second, and the second is what sets the loss.

4.2 From five updates to a longer training budget

Setup for every new run: 153.6M tokens, batch 64, AdamW $\beta = (0.9, 0.95)$, weight decay 0.1 (held fixed), seed 42, same data order. The baseline is the course default (Kaiming style, identical to μP at the reference width 512); μP changes only the four table settings with $n_0 = 512$. The supplied width 512 curve is the source (best sampled LR 0.003, fitted 0.0024).

(a)–(b) Width transfer (20 new runs, five LRs per cell)

Prediction before running: 0.003 transfers to widths 128 and 256 under μP (loss at 0.003 within noise of the tuned loss) but not under the baseline, whose optimum should rise as width shrinks.

policy	width	best sampled lr	fitted lr*	tuned loss (fit)	loss at transferred lr 0.003	gap to best sampled
baseline	128	0.006	0.0074	3.613	3.693	0.067
baseline	256	0.003	0.0048	3.388	3.405	0
μP	128	0.003	0.0037	3.628	3.628	0
μP	256	0.0015	0.0025	3.406	3.419	0.006
source (both)	512	0.003	0.0024	3.224	3.229	0

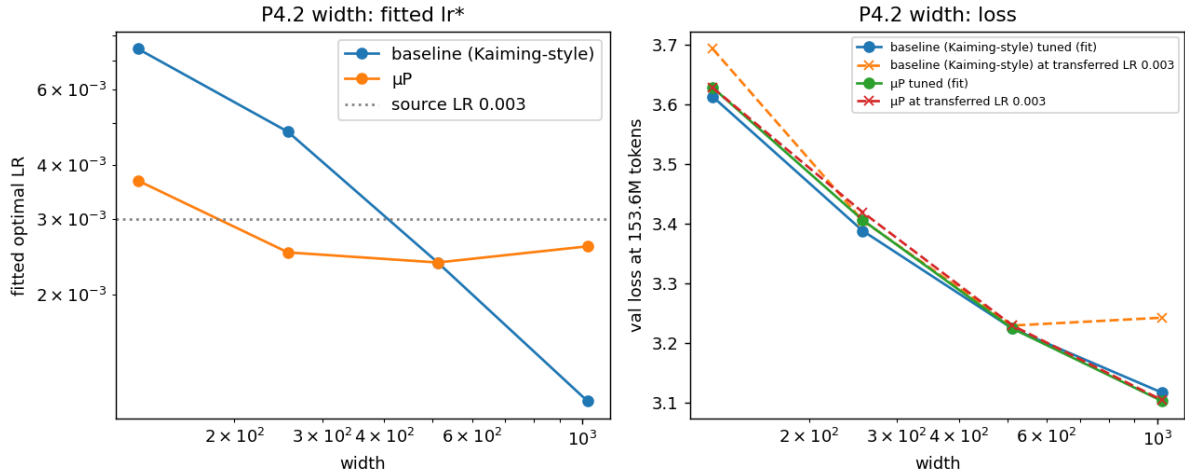


Figure 10: Fitted optimal LR vs. width (left; dotted = source LR 0.003) and tuned vs. transferred-LR loss (right), both prescriptions, including the held-out width 1024 from part (c).

Power laws through the three fitted optima, recorded before part (c): baseline $lr^* \propto \text{width}^{-0.82}$, which predicts 0.0014 at 1024; μP $lr^* \propto \text{width}^{-0.31}$, which predicts 0.0018 at 1024.

Transfer: μP still wins, but less cleanly than in five steps. From 512 to 128 the baseline optimum moves $3\times$ ($0.0024 \rightarrow 0.0074$) and μP ’s moves $1.5\times$ (to 0.0037). At the transferred learning rate the baseline loses 0.067 at width 128; μP loses nothing measurable. But μP ’s optimum is not flat (exponent -0.31), so over a $4\times$ width change “tune once” only approximately holds.

Performance: the tuned baseline is slightly better, 3.613 / 3.388 against μP ’s 3.628 / 3.406, ahead by 0.015 to 0.018 at both widths. In the five step test μP was ahead by 0.2 to 0.7.

Why it flips: in five steps only the size of the first updates mattered. Over 2,444 updates with warmup and decay, a $3\times$ mis-sized learning rate is survivable (the baseline at 0.003 is 0.067 behind its own optimum), and two effects

work against μP at narrow widths. The $1/m$ readout multiplier becomes a $4\times$ boost at width 128 (initial logit RMS 1.96 vs. 0.98). And the hidden learning rate is multiplied by 4 while weight decay stays at 0.1, so the $\text{lr} \cdot \text{wd}$ product that Problem 2 showed controls the averaging window is $4\times$ the source value. μP transfers the step size but not the regularisation timescale. Holding weight decay fixed is a hidden mistuning that grows with the width ratio.

(c) **Held-out width 1024 (8 new runs)**

policy	lr 0.00075	power-law pick (0.0014 / 0.0018)	lr 0.0015	lr 0.003 (direct transfer)	fitted lr*	fitted loss
baseline	3.139	3.114	3.135	3.242	0.0011	3.116
μP	3.240	3.111	3.124	3.104	0.0026	3.102

Gaps to the best sampled point: for the baseline, the power law pick is 0 off and direct transfer is $+0.128$; for μP , direct transfer is 0 off and the power law pick is $+0.007$. Fitting the width dependence beats direct transfer for the baseline by 0.128 and slightly hurts μP . The baseline's optimum moved to a third of the source value, as $\text{width}^{-0.82}$ said it would. μP 's extrapolated exponent over corrected because its drift happened at the narrow widths (128, 256), not between 512 and 1024. That fits the fixed weight decay and decaying α mechanisms being largest when the width ratio is far from 1 in the shrinking direction. Once each is tuned, the two reach the same place (3.114 vs. 3.104).

A caveat: the baseline's 1024 curve is not smooth at the 0.02 level ($0.0014 \rightarrow 3.114$ vs. $0.0015 \rightarrow 3.135$, a gap present from step 300 on), so single runs at this width resolve learning rate differences of about 0.02 at best. The 0.128 and 0.067 headline gaps stand; the 0.007 μP gap does not.

(d) **Depth transfer at width 512 (18 new runs; reference depth 8, $r = \text{depth}/8$)**

prescription	depth	best sampled lr	fitted lr*	tuned loss (fit)	loss at transferred lr 0.003	gap
μP (no depth rule)	4	0.003	0.0025	3.312	3.314	0
Depth- μP	4	0.0015	0.0016	3.326	3.344	0.018
CompleteP	4	0.003	0.0025	3.313	3.316	0
source	8	0.003	0.0024	3.224	3.229	0
μP (no depth rule)	16	0.0015	0.0021	3.167	3.175	0.002
Depth- μP	16	0.003	0.0028	3.169	3.169	0
CompleteP	16	0.0015	0.0017	3.169	3.193	0.023

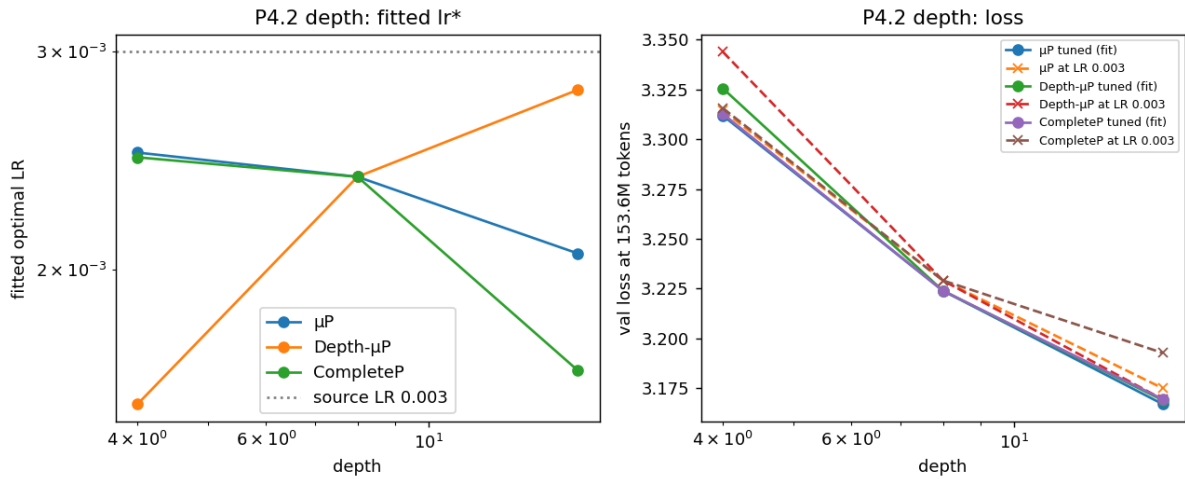


Figure 11: Fitted optimal LR vs. depth (left) and tuned vs. transferred-LR loss (right) for μP , Depth- μP , CompleteP at width 512.

Over a $4\times$ depth range the depth corrections change nothing the data can resolve. Tuned losses agree within 0.003 at depth 16 and 0.014 at depth 4 (seed noise is 0.002 to 0.005), and the fitted optima wander between 0.0016 and

0.0028 with no consistent ordering: Depth- μ P misses at depth 4, CompleteP at depth 16. With r between $\frac{1}{2}$ and 2 the residual stream grows by at most about $1.4\times$, which the final RMSNorm and warmup absorb, unlike depth 100 to 1000 in P4.1. The depth rules are insurance for extreme depth, not a tuning knob at ordinary depth. Here they improve neither transfer nor tuned loss.

(e) **Training-regime diagnostics (fixed inputs, source LR 0.003)**

configuration	α_{upd} hidden (median), step 5 $\rightarrow$ 100 $\rightarrow$ end	α_{upd} readout, step 5 $\rightarrow$ end	ω_{move}	logit RMS, init $\rightarrow$ end	feature movement, step 5 $\rightarrow$ end
baseline w128 d8	0.79 $\rightarrow$ 0.73 $\rightarrow$ 0.61	0.87 $\rightarrow$ 0.64	0.50	0.98 $\rightarrow$ 4.4	1.29 $\rightarrow$ 2.02
baseline w256 d8	0.90 $\rightarrow$ 0.73 $\rightarrow$ 0.61	0.95 $\rightarrow$ 0.63	0.50	0.99 $\rightarrow$ 4.2	1.20 $\rightarrow$ 1.63
μ P w128 d8	0.88 $\rightarrow$ 0.71 $\rightarrow$ 0.60	0.92 $\rightarrow$ 0.60	0.50	1.96 $\rightarrow$ 4.4	1.37 $\rightarrow$ 1.26
μ P w256 d8	0.91 $\rightarrow$ 0.74 $\rightarrow$ 0.61	0.94 $\rightarrow$ 0.62	0.50	1.40 $\rightarrow$ 4.2	1.25 $\rightarrow$ 1.22
μ P / Depth- μ P / CompleteP w512 d16	0.91 $\rightarrow$ 0.77– 0.78 $\rightarrow$ 0.63–0.64	0.96 $\rightarrow$ 0.63	0.50	0.98 $\rightarrow$ 4.1–4.3	1.30 $\rightarrow$ 1.34–1.35

Gradient clipping (max norm 1) is active only at the start. The step 1 clip coefficient is 0.26 (baseline, width 128) and 0.11 (baseline, width 256) versus 0.034 and 0.040 under μ P, whose larger readout multiplier makes the initial gradient 3 to $8\times$ larger. It is identical across learning rates because the first gradient does not depend on the learning rate. After the first few dozen updates the coefficient is 1 at every logged point in every configuration, so clipping shapes the first steps and nothing after.

Update alignment decays during training. $\alpha \approx 0.9$ at step 5, near the derivation’s $\alpha = 1$; 0.73 to 0.78 at step 100; 0.60 to 0.64 at the end, only a little above random ($\frac{1}{2}$). Early training is a coherent, nearly rank one push; late training is closer to noise averaging. The η/m rule is derived for $\alpha = 1$, so it is most exact in the phase the five step test measured and increasingly over conservative later. That is consistent with μ P’s optimum drifting *up* at narrow widths over a full run.

Readout to feature change alignment never appears: $\omega_{\text{move}} = 0.50$ at every width, depth, prescription and step. So $\alpha = 1$ holds early and fails late, and $\omega = 1$ holds nowhere. The rules that depend on α still work approximately; the one that depends on ω is irrelevant to the learning rate.

Transients do not decide the final loss. μ P at width 128 starts with logits twice as large (1.96 vs. 0.98) and moves its features further in five steps (1.37 vs. 1.29), yet its loss at the transferred learning rate is *better* (3.628 vs. 3.693). The baseline’s features end up moving the most (2.02) and it is worst. The step size decided the outcome, not the size of the early transients.

Does a shifted optimum mean the model is outside the stable μ P regime? No. μ P’s optimum moved $1.5\times$ between widths 512 and 128 while every stability signal stayed bounded: logit RMS grows the same way for all configurations, feature movement is 1.2 to 1.4, and the loss curves are smooth and flat. A drifting optimum with bounded diagnostics points to a mis-scaled *secondary* quantity. Here that is the fixed weight decay (Problem 2’s coupling: μ P multiplies the hidden learning rate by $1/m$ but leaves $\text{lr} \cdot \text{wd}$ unscaled, so the averaging window changes by m) together with the decaying α , not a breakdown of feature learning.

Transferring a learning rate *upward* in width works as-is with μ P (0.003 was the best point at 1024). With a standard parameterisation it only works through a fitted power law; direct transfer cost 0.128. After tuning, the two reach the same loss to within 0.01 to 0.02, so over a real budget μ P buys predictability rather than performance. That is the opposite emphasis from the five step test, where it bought both. Depth corrections are irrelevant at depths 4 to 16. And whenever the width ratio exceeds about $2\times$, weight decay needs another look: the $\text{lr} \cdot \text{wd}$ timescale is the one quantity none of these prescriptions preserves, and it is the most likely cause of μ P’s remaining optimum drift at narrow widths.